

```

1 -----
2 -- A single stage of a parallel CORDIC calculator.
3 --
4 -- This entity describes a single stage of a CORDIC calculator. It includes
5 -- the X, Y, and Z adders.
6
7 -- Revision History:
8 --   Date:           Author:      Description:
9 --   18 Feb 25       Chris M.      Initial revision.
10 --   21 Feb 25       Chris M.      Changed i from input to generic.
11 --   25 Feb 25       Chris M.      Added documentation.
12 --
13 -- TODO:
14 --   - Internally generate the d signal.
15 --
16 -----
17 library ieee;
18 use std.textio.all;
19 use ieee.std_logic_1164.all;
20 use ieee.numeric_std.all;
21 use ieee.math_real.all;
22 use work.FixedLib.all;
23
24 -- This must be defined here to avoid circular dependencies.
25 package CordicConstants is
26
27 -- Function constants. Defined in the spec.
28 --   1  = cos(x)
29 --   2  = cosh(x)
30 --   4  = x*y
31 --   5  = sin(x)
32 --   6  = sinh(x)
33 --  12  = y/x
34 constant F_COS_X      : std_logic_vector(4 downto 0) := "00001";
35 constant F_SIN_X      : std_logic_vector(4 downto 0) := "00101";
36 constant F_X_MULT_Y   : std_logic_vector(4 downto 0) := "00100";
37 constant F_COSH_X     : std_logic_vector(4 downto 0) := "00010";
38 constant F_SINH_X     : std_logic_vector(4 downto 0) := "00110";
39 constant F_Y_DIV_X    : std_logic_vector(4 downto 0) := "01100";
40
41 -- Coordinate systems. Trig functions are done in the circular coordinate
42 -- system, arithmetic functions are done in the linear coordinate system, and
43 -- hyperbolic functions are done in the hyperbolic coordinate system.
44 signal    CordSystem   : std_logic_vector(1 downto 0);
45 constant CIRCULAR     : std_logic_vector(1 downto 0) := "01"; -- (1)
46 constant LINEAR       : std_logic_vector(1 downto 0) := "00"; -- (0)
47 constant HYPERBOLIC   : std_logic_vector(1 downto 0) := "11"; -- (-1)
48
49 -- The number of iterations.
50 constant ITERATIONS    : positive := 16;
51 constant INTERMEDIATE_WORD_SIZE : positive := 22;
52 constant WORD_SIZE     : positive := 16;
53
54 -- Starting constants for circular and hyperbolic modes.

```

```

55  --
56  constant K_inv_circular    : std_logic_vector((INTERMEDIATE_WORD_SIZE - 1) downto 0) :=
57      RealToQ3_18(0.607252935);
58
59  constant K_inv_hyperbolic : std_logic_vector((INTERMEDIATE_WORD_SIZE - 1) downto 0) :=
60      RealToQ3_18(1.20749706776);
61
62  constant ZERO_22 : std_logic_vector((INTERMEDIATE_WORD_SIZE - 1) downto 0) := (others => '0');
63
64  -- 17 x 22 matrix for the generating busses. Note that intermediate operations
65  -- will use Q3.18 fixed points to avoid overflow rounding errors.
66  type array_2d is array (0 to (ITERATIONS - 1)) of
67      std_logic_vector((INTERMEDIATE_WORD_SIZE - 1) downto 0);
68
69  -- Constants for the B inputs of the Z adders.
70  --
71  constant Z_CONSTANTS_CIRCULAR    : array_2d := (
72      -- a_i = arctan(2^-i) for 0 <= i <= 15
73      RealToQ3_18(arctan(2.0**(0))),
74      RealToQ3_18(arctan(2.0**(-1))),
75      RealToQ3_18(arctan(2.0**(-2))),
76      RealToQ3_18(arctan(2.0**(-3))),
77      RealToQ3_18(arctan(2.0**(-4))),
78      RealToQ3_18(arctan(2.0**(-5))),
79      RealToQ3_18(arctan(2.0**(-6))),
80      RealToQ3_18(arctan(2.0**(-7))),
81      RealToQ3_18(arctan(2.0**(-8))),
82      RealToQ3_18(arctan(2.0**(-9))),
83      RealToQ3_18(arctan(2.0**(-10))),
84      RealToQ3_18(arctan(2.0**(-11))),
85      RealToQ3_18(arctan(2.0**(-12))),
86      RealToQ3_18(arctan(2.0**(-13))),
87      RealToQ3_18(arctan(2.0**(-14))),
88      RealToQ3_18(arctan(2.0**(-15)))
89  );
90
91  constant Z_CONSTANTS_LINEAR      : array_2d := (
92      -- a_i = 2^-i for 0 <= i <= 15
93      RealToQ3_18(2.0**(0)),
94      RealToQ3_18(2.0**(-1)),
95      RealToQ3_18(2.0**(-2)),
96      RealToQ3_18(2.0**(-3)),
97      RealToQ3_18(2.0**(-4)),
98      RealToQ3_18(2.0**(-5)),
99      RealToQ3_18(2.0**(-6)),
100     RealToQ3_18(2.0**(-7)),
101     RealToQ3_18(2.0**(-8)),
102     RealToQ3_18(2.0**(-9)),
103     RealToQ3_18(2.0**(-10)),
104     RealToQ3_18(2.0**(-11)),
105     RealToQ3_18(2.0**(-12)),
106     RealToQ3_18(2.0**(-13)),
107     RealToQ3_18(2.0**(-14)),
108     RealToQ3_18(2.0**(-15))

```

```

109 );
110
111 -- Note that we start at iteration 1 because arctanh(2^0) is undefined, and
112 -- we repeat iterations 4 and 13 so that the expression
113 --
114 --  $a_1 - \sum_{k=2}^{16} a_k$ 
115 --
116 -- converges to zero. See Walther, 1971 for more details.
117 constant Z_CONSTANTS_HYPERBOLIC : array_2d := (
118     --  $a_i = \text{arctanh}(2^{-k})$  for  $0 \leq i \leq 15$  where
119     --
120     -- { i + 1 : i < 4
121     -- k = { i : 4 <= i <= 13
122     -- { i - 1 : i > 13
123     --
124     RealToQ3_18(arctanh(2.0**(-1))), -- a_1
125     RealToQ3_18(arctanh(2.0**(-2))), -- a_2
126     RealToQ3_18(arctanh(2.0**(-3))), -- a_3
127     RealToQ3_18(arctanh(2.0**(-4))), -- a_4
128     RealToQ3_18(arctanh(2.0**(-4))), -- a_5 (repeat)
129     RealToQ3_18(arctanh(2.0**(-5))), -- a_6
130     RealToQ3_18(arctanh(2.0**(-6))), -- a_7
131     RealToQ3_18(arctanh(2.0**(-7))), -- a_8
132     RealToQ3_18(arctanh(2.0**(-8))), -- a_9
133     RealToQ3_18(arctanh(2.0**(-9))), -- a_10
134     RealToQ3_18(arctanh(2.0**(-10))), -- a_11
135     RealToQ3_18(arctanh(2.0**(-11))), -- a_12
136     RealToQ3_18(arctanh(2.0**(-12))), -- a_13
137     RealToQ3_18(arctanh(2.0**(-13))), -- a_14
138     RealToQ3_18(arctanh(2.0**(-13))), -- a_15 (repeat)
139     RealToQ3_18(arctanh(2.0**(-14))) -- a_16
140 );
141
142 -- String arrays for debugging using value as index.
143 type ModeStringType is array(0 to 1) of string;
144 constant MODE_STRINGS : ModeStringType := ( "Rotation ", "Vectoring" );
145
146 type CordStringType is array(0 to 3) of string;
147 constant CORD_STRINGS : CordStringType := (
148     "Linear ",
149     "Circular ",
150     " ",
151     "Hyperbolic"
152 );
153
154
155 end package;
156
157
158 library ieee;
159 use std.textio.all;
160 use ieee.std_logic_1164.all;
161 use ieee.numeric_std.all;
162 use ieee.math_real.all;

```

```

163 use work.FixedLib.all;
164 use work.CordicConstants.all;
165 use work.all;
166
167
168 -- CordicStage entity declaration.
169 --
170 -- This entity describes a stage of a fully parallel cordic. A stage consists
171 -- of X, Y, and Z adders, and has a fixed right shift amount determined at compile time.
172 -- Note that only one input (the A input) per adder is needed, since the B for either the X
173 -- or Y adder is the A input for the opposite adder shifted right i times. Finally, note that
174 -- iteration repetitions do assure convergence for hyperbolic functions is only valid for i < 40.
175 -- See Walther (1971) for more info.
176 --
177 -- Generics:
178 --
179 --     wordsize : The size of the inputs.
180 --     i         : The index of this CordicStage. Determines the shift amounts and iteration
181 --                 repetitions.
182 -- Inputs:
183 --
184 --     XIn       : The A input for the X adder. X >> i is the B input for the Y adder.
185 --     YIn       : The B input for the Y adder. Y >> i is the B input for the X adder.
186 --     ZIn       : The A input for the Z adder.
187 --     ZConstant  : The B input for the Z adder.
188 --     CordSystem : The coordinate system this stage is operating in (Circular, Linear, or
189 --                 Hyperbolic). Defined in the CordicConstants package.
190 --     d         : The decision variable for this adder. Used to generate the Add/BarSub signals
191 --                 for each adder. Note that because these only depend on the inputs to the
192 --                 current stage, they can be generated internally (and should be later).
193 --
194 -- Outputs:
195 --
196 --     XOut       : The output of the X adder.
197 --     YOut       : The output of the Y adder.
198 --     ZOut       : The output of the Z adder.
199 --
200 entity CordicStage is
201   generic (
202     wordsize : positive := 22;
203     i         : positive range 0 to wordsize
204   );
205
206   port(
207     XIn       : in std_logic_vector(wordsize - 1 downto 0);
208     YIn       : in std_logic_vector(wordsize - 1 downto 0);
209     ZIn       : in std_logic_vector(wordsize - 1 downto 0);
210     ZConstant  : in std_logic_vector(wordsize - 1 downto 0);
211     CordSystem : in std_logic_vector(1 downto 0);
212     d         : in std_logic;
213
214     XOut       : out std_logic_vector(wordsize - 1 downto 0);
215     YOut       : out std_logic_vector(wordsize - 1 downto 0);
216     ZOut       : out std_logic_vector(wordsize - 1 downto 0)

```

```

217 );
218 end CordicStage;
219
220 architecture concurrent of CordicStage is
221
222     signal XAddBarSub : std_logic;
223     signal YAddBarSub : std_logic;
224     signal ZAddBarSub : std_logic;
225
226     -- Used the pass the X Input through when the coordinate system is linear.
227     signal XOutMux : std_logic_vector(XIn'range);
228
229     -- `m` as shown in Walther's paper. The Cordinate System constants were picked
230     --- such that the high bit corresponds to it's sign.
231     alias Mode : std_logic is CordSystem(1);
232
233     -- B inputs for adders.
234     signal X_BIn : std_logic_vector(XIn'range);
235     signal Y_BIn : std_logic_vector(XIn'range);
236
237 begin
238
239     -- Bypass the output when in the linear coordinate system.
240     XOut <= XIn      when (CordSystem = LINEAR) else
241             XOutMux;
242
243     -- Determine the AddBarSub signals
244     -- m d_i
245     -- If d_i = 0 (Add), then we want to subtract
246     XAddBarSub <= d xnor Mode;
247     YAddBarSub <= d;
248     ZAddBarSub <= d xor '1';
249
250     -- Generate the B inputs for the adders. Note that we must special case the sequence of indices
251     -- if Cordinate System is hyperbolic. Specifically start at iteration 1 because arctanh(2**0) =
252     -- arctanh(1) is undefined. Additionally, we must repeat iterations 4 and 13 because otherwise
253     --
254     --  $a_1 - \sum_{k=2}^{16} a_n$ 
255     --
256     -- where a_i are the Z constants for the nth iteration, does not converge to zero. Thus, we obtain
257     -- a mapping from i, the number of the adder, and k, the number of shifts we want to do:
258     --
259     -- { i + 1 : i < 4
260     -- k = { i : 4 <= i <= 13
261     -- { i = 1 : i > 13
262     --
263     -- Finally. Note that the ith constant is defined by
264     --
265     --  $a_i = \text{arctanh}(2^{-k})$ 
266     --
267     -- For 0 <= i <= 15
268     --
269     -- The iteration sequence is as follows for hyperbolic functions:
270     --

```

```

271  -- 0 -> 1 (+ 1)
272  -- 1 -> 2
273  -- 2 -> 3
274  -- 3 -> 4
275  -- 4 -> 4 ( 1 : 1 )
276  -- 5 -> 5
277  -- 6 -> 6
278  -- 7 -> 7
279  -- 8 -> 8
280  -- 9 -> 9
281  -- 10 -> 10
282  -- 11 -> 11
283  -- 12 -> 12
284  -- 13 -> 13
285  -- 14 -> 13 ( - 1)
286  -- 15 -> 14
287  X_BIn <= std_logic_vector( shift_right( signed(Yin), (i + 1) ) ) when ((CordSystem = HYPERBOLIC) and (i <
4)) else
288      std_logic_vector( shift_right( signed(Yin), (i - 1) ) ) when ((CordSystem = HYPERBOLIC) and (i >
13)) else
289      std_logic_vector( shift_right( signed(Yin), i));
290
291  Y_BIn <= std_logic_vector( shift_right( signed(Xin), (i + 1) ) ) when ((CordSystem = HYPERBOLIC) and (i <
4)) else
292      std_logic_vector( shift_right( signed(Xin), (i - 1) ) ) when ((CordSystem = HYPERBOLIC) and (i >
13)) else
293      std_logic_vector( shift_right( signed(Xin), i));
294
295
296  -- The A input for X adders is the X Input, while the B input is the Y Input
297  -- shifted right by i.
298  XAdder : entity AddSub
299      generic map (wordsize)
300      port map (
301          A          => XIn,
302          B          => X_BIn,
303          Cin        => XAddBarSub,
304          AddBarSub  => XAddBarSub,
305          Cout       => open,          -- Carry out is unused. Might cause warnings in Xilinx ISE.
306          S          => XOutMux
307      );
308
309  -- The A input for Y adders is the Y input, while the B input is the X input
310  -- shifted right by i.
311  YAdder : entity AddSub
312      generic map (wordsize)
313      port map (
314          A          => YIn,
315          B          => Y_BIn,
316          Cin        => YAddBarSub,
317          AddBarSub  => YAddBarSub,
318          Cout       => open,
319          S          => YOut
320      );

```

```
321
322  ZAdder : entity AddSub
323    generic map (wordsize)
324    port map (
325      A          => ZIn,
326      B          => ZConstant,
327      Cin        => ZAddBarSub,
328      AddBarSub  => ZAddBarSub,
329      Cout       => open,
330      S          => ZOut
331    );
332
333 end concurrent;
334
```